

LAPORAN PRATIKUM PEKAN 8
STRUKTUR DATA

Disusun Oleh:

Rifqi Aditya

2511533002

Dosen Pengampu :

Dr. Wahyudi, S.T, M.T

Asisten Praktikum :

Sherly Sukmadira



DEPARTEMEN INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS ANDALAS
2026

KATA PENGANTAR

Puji syukur ke hadirat Allah SWT karena atas rahmat dan karunia-Nya saya dapat menyelesaikan laporan praktikum mata Struktur Data tentang *Sorting*.

Laporan ini disusun untuk memenuhi salah satu tugas praktikum sekaligus sebagai sarana pembelajaran dalam memahami dasar-dasar pemrograman menggunakan bahasa Java. Melalui praktikum ini, penulis dapat mempelajari penggunaan *Sorting*.

Penulis menyadari bahwa laporan ini masih jauh dari sempurna. Oleh karena itu, penulis mengucapkan terima kasih kepada dosen pengampu, asisten praktikum, serta rekan-rekan yang telah membantu dalam proses penyusunan laporan ini.

Akhir kata, semoga laporan ini dapat memberikan manfaat, baik bagi penulis sendiri maupun bagi pembaca, khususnya dalam memperdalam pemahaman tentang pemrograman dasar dengan Java.

Padang, 6 Juni 2025

Penulis

Daftar Pustaka

BAB I PENDAHULUAN.....	4
1.1 Latar Belakang	4
1.2 Tujuan	4
1.3 Manfaat.....	4
BAB II PEMBAHASAN.....	5
2.1 Shell Sort.....	5
2.1.1 Kode Program	5
2.1.2 Uraian Kode Program.....	5
2.1.3 Output	6
2.2 Quick Sort	6
2.2.1 Kode Program	6
2.2.2 Uraian Kode Program.....	7
2.2.3 Output	7
2.3 Merge Sort	8
2.3.1 Kode Program	8
2.3.2 Uraian Kode Program.....	9
2.3.3 Output	10
2.4 Merge Sort GUI.....	10
2.4.1 Kode Program	10
2.4.2 Uraian Kode Program.....	12
2.4.3 Output	14
2.5 Bubble Sort GUI.....	14
2.5.1 Kode Program	14
2.5.2 Uraian Kode Program.....	16
2.5.3 Output	17
BAB III TUGAS	18
3.1 Playlist Lagu.....	18
3.1.1 Kode Program	18
3.1.2 Uraian Kode Program.....	18
3.2 Output.....	19
BAB III KESIMPULAN	20

BAB I

PENDAHULUAN

1.1 Latar Belakang

Sorting atau pengurutan data merupakan salah satu materi penting dalam struktur data dan algoritma. Sorting digunakan untuk menyusun data berdasarkan urutan tertentu, misalnya dari nilai terkecil ke terbesar atau sebaliknya. Dalam pemrograman, proses sorting sangat sering digunakan, terutama ketika data yang dikelola berjumlah banyak dan perlu disusun agar lebih mudah dicari, dibaca, maupun dianalisis.

Pada praktikum ini dipelajari beberapa algoritma sorting, yaitu Quick Sort, Shell Sort, Merge Sort, Merge Sort GUI, dan Bubble Sort GUI. Setiap algoritma memiliki cara kerja yang berbeda-beda. Quick Sort menggunakan konsep pembagian data berdasarkan pivot, Shell Sort menggunakan perbandingan dengan jarak atau gap tertentu, Merge Sort menggunakan metode membagi dan menggabungkan data, sedangkan Bubble Sort membandingkan data secara berpasangan. Dengan adanya versi GUI, proses sorting dapat divisualisasikan sehingga lebih mudah dipahami.

1.2 Tujuan

1. Memahami konsep dasar sorting atau pengurutan data.
2. Mempelajari cara kerja berbagai algoritma sorting.
3. Membandingkan proses dari setiap algoritma sorting.

1.3 Manfaat

1. Meningkatkan pemahaman tentang algoritma sorting.
2. Mempermudah pemahaman proses sorting melalui tampilan GUI.
3. Membantu membandingkan efisiensi algoritma sorting.

BAB II

PEMBAHASAN

2.1 Shell Sort

2.1.1 Kode Program

```
1 package Pekan8_2511533002;
2
3 public class ShellSort_2511533002 {
4     public static void shellSort_3002(int[] A) {
5         int n_3002 = A.length;
6         int gap_3002 = n_3002 / 2;
7         while (gap_3002 > 0) {
8             for (int i_3002 = gap_3002; i_3002 < n_3002; i_3002++) {
9                 int temp_3002 = A[i_3002];
10                int j_3002 = i_3002;
11                while (j_3002 >= gap_3002 && A[j_3002 - gap_3002] > temp_3002) {
12                    A[j_3002] = A[j_3002 - gap_3002];
13                    j_3002 = j_3002 - gap_3002;
14                }
15                A[j_3002] = temp_3002;
16            }
17            gap_3002 = gap_3002 / 2;
18        }
19    }
20
21    public static void main(String[] args) {
22        int[] data_3002 = {3, 10, 4, 6, 8, 9, 7, 2, 1, 5};
23
24        System.out.print("Sebelum: ");
25        printArray_3002(data_3002);
26
27        shellSort_3002(data_3002);
28        System.out.print("Sesudah (Shell Sort): ");
29        printArray_3002(data_3002);
30    }
31
32    public static void printArray_3002(int[] arr_3002) {
33        for (int i : arr_3002) System.out.print(i + " ");
34        System.out.println();
35    }
36 }
```

2.1.2 Uraian Kode Program

1. **int n_3002 = A.length;** = Menghitung jumlah data.
2. **int gap_3002 = n_3002 / 2;** = Menentukan nilai gap awal
3. **while (gap_3002 > 0) {** = Perulangan selama gap masih lebih dari 0
4. **for (int i_3002 = gap_3002; i_3002 < n_3002; i_3002++) {** = Perulangan dari indeks gap sampai akhir array
5. **int temp_3002 = A[i_3002];** = Menyimpan elemen sementara
6. **int j_3002 = i_3002;** = Menyimpan posisi awal elemen
7. **while (j_3002 >= gap_3002 && A[j_3002 - gap_3002] > temp_3002) {** = Membandingkan dan menggeser elemen
8. **A[j_3002] = A[j_3002 - gap_3002];** = Menggeser elemen
9. **j_3002 = j_3002 - gap_3002;** = Mengurangi posisi j
10. **A[j_3002] = temp_3002;** = Menempatkan nilai sementar
11. **gap_3002 = gap_3002 / 2;** = Mengecilkan nilai gap
12. **int[] data_3002 = {3, 10, 4, 6, 8, 9, 7, 2, 1, 5};** = membuat data array
for (int i : arr_3002) System.out.print(i + " "); = Perulangan mencetak isi array

2.1.3 Output

```
Sebelum: 3 10 4 6 8 9 7 2 1 5
Sesudah (Shell Sort): 1 2 3 4 5 6 7 8 9 10
```

2.2 Quick Sort

2.2.1 Kode Program

```
1 package Pekan8_2511533002;
2
3 public class QuickSort_2511533002 {
4
5     static void swap(int[] arr_3002, int i_3002, int j_3002) {
6         int temp_3002 = arr_3002[i_3002];
7         arr_3002[i_3002] = arr_3002[j_3002];
8         arr_3002[j_3002] = temp_3002;
9     }
10
11     // Metode tambahan untuk mengatur pivot menggunakan Median-of-Three
12     static void medianOfThree(int[] arr_3002, int low_3002, int high_3002) {
13         int mid_3002 = low_3002 + (high_3002 - low_3002) / 2;
14
15         // Urutkan elemen low, mid, dan high
16         if (arr_3002[low_3002] > arr_3002[mid_3002]) {
17             swap(arr_3002, low_3002, mid_3002);
18         }
19         if (arr_3002[low_3002] > arr_3002[high_3002]) {
20             swap(arr_3002, low_3002, high_3002);
21         }
22         if (arr_3002[mid_3002] > arr_3002[high_3002]) {
23             swap(arr_3002, mid_3002, high_3002);
24         }
25         swap(arr_3002, mid_3002, high_3002);
26     }
27
28     static int partition(int[] arr_3002, int low_3002, int high_3002) {
29         // Panggil fungsi medianOfThree sebelum menentukan pivot
30         medianOfThree(arr_3002, low_3002, high_3002);
31
32         int pivot_3002 = arr_3002[high_3002]; // Sekarang arr[high] sudah berisi nilai median
33         int i_3002 = (low_3002 - 1);
34
35         for (int j_3002 = low_3002; j_3002 <= high_3002 - 1; j_3002++) {
36             // Jika elemen saat ini lebih kecil dari atau sama dengan pivot
37             if (arr_3002[j_3002] < pivot_3002) {
38                 // Increment indeks elemen yang lebih kecil
39                 i_3002++;
40                 swap(arr_3002, i_3002, j_3002);
41             }
42         }
43         swap(arr_3002, i_3002 + 1, high_3002);
44         return (i_3002 + 1);
45     }
46     static void quickSort(int[] arr_3002, int low_3002, int high_3002)
47     {
48         if (low_3002 < high_3002) {
49             int pi_3002 = partition(arr_3002, low_3002, high_3002);
50             quickSort(arr_3002, low_3002, pi_3002 - 1);
51             quickSort(arr_3002, pi_3002 + 1, high_3002);
52         }
53     }
54
55     public static void printArr(int[] arr_3002)
56     {
57         for (int i_3002 = 0; i_3002 < arr_3002.length; i_3002++) {
58             System.out.print(arr_3002[i_3002] + " ");
59         }
60         System.out.println();
61     }
62
63     public static void main(String[] args)
64     {
65         int[] arr_3002 = { 10, 7, 8, 9, 1, 5 };
66         int N_3002 = arr_3002.length;
67         System.out.print("Data sebelum diurutkan: ");
68         printArr(arr_3002);
69
70         quickSort(arr_3002, 0, N_3002 - 1);
71
72         System.out.print("Data Terurut quicksort: ");
73         printArr(arr_3002);
74     }
75 }
```

2.2.2 Uraian Kode Program

1. **static void swap(int[] arr_3002, int i_3002, int j_3002) {**
 int temp_3002 = arr_3002[i_3002];
 arr_3002[i_3002] = arr_3002[j_3002];
 arr_3002[j_3002] = temp_3002; = digunakan untuk menukar posisi dua elemen dalam array.
2. **static void medianOfThree(int[] arr_3002, int low_3002, int high_3002) {**
 int mid_3002 = low_3002 + (high_3002 - low_3002) / 2; = digunakan untuk menentukan pivot dengan teknik Median-of-Three.
3. **int mid_3002 = low_3002 + (high_3002 - low_3002) / 2;** = digunakan untuk mencari indeks tengah dari array.
4. **static int partition(int[] arr_3002, int low_3002, int high_3002)** = Method ini digunakan untuk membagi array menjadi dua bagian berdasarkan pivot.
5. **for (int j_3002 = low_3002; j_3002 <= high_3002 - 1; j_3002++)** = Perulangan ini digunakan untuk mengecek setiap elemen dari indeks awal sampai sebelum pivot.
6. **swap(arr_3002, i_3002 + 1, high_3002);**
 return (i_3002 + 1); = Nilai yang dikembalikan adalah posisi akhir pivot setelah proses partition.
7. **if (low_3002 < high_3002)** = Kondisi ini memastikan bahwa proses pengurutan hanya dilakukan jika masih ada lebih dari satu elemen.
8. **int pi_3002 = partition(arr_3002, low_3002, high_3002);** = Program memanggil method **partition** untuk menentukan posisi pivot, setelah pivot berada di posisi yang benar, array dibagi menjadi dua bagian
9. **for (int i_3002 = 0; i_3002 < arr_3002.length; i_3002++)** = Perulangan ini berjalan dari indeks pertama sampai indeks terakhir array.
10. **quickSort(arr_3002, 0, N_3002 - 1);** = Program memanggil method quickSort untuk mengurutkan array dari indeks pertama sampai indeks terakhir.

2.2.3 Output

```
Data sebelum diurutkan: 10 7 8 9 1 5
Data Terurut quicksort: 1 5 7 8 9 10
```

2.3 Merge Sort

2.3.1 Kode Program

```
1 package Pekan8_2511533002;
2
3 public class MergeSort_2511533002 {
4     void merge(int arr_3002[], int l_3002, int m_3002, int r_3002) {
5         // Find sizes of two subarrays to be merged
6         int n1_3002 = m_3002 - l_3002 + 1;
7         int n2_3002 = r_3002 - m_3002;
8
9         /* Create temp arrays */
10        int L_3002[] = new int[n1_3002];
11        int R_3002[] = new int[n2_3002];
12
13        /* Copy data to temp arrays */
14        for (int i_3002 = 0; i_3002 < n1_3002; ++i_3002)
15            L_3002[i_3002] = arr_3002[l_3002 + i_3002];
16        for (int j_3002 = 0; j_3002 < n2_3002; ++j_3002)
17            R_3002[j_3002] = arr_3002[m_3002 + 1 + j_3002];
18
19        int i_3002 = 0, j_3002 = 0;
20
21        // Initial index of merged subarray array
22        int k_3002 = l_3002;
23        while (i_3002 < n1_3002 && j_3002 < n2_3002) {
24            if (L_3002[i_3002] <= R_3002[j_3002]) {
25                arr_3002[k_3002] = L_3002[i_3002];
26                i_3002++;
27            }
28            else {
29                arr_3002[k_3002] = R_3002[j_3002];
30                j_3002++;
31            }
32            k_3002++;
33        }
34
35        /* Copy remaining elements of L[] if any */
36        while (i_3002 < n1_3002) {
37            arr_3002[k_3002] = L_3002[i_3002];
```

```
38            i_3002++;
39            k_3002++;
40        }
41
42        /* Copy remaining elements of R[] if any */
43        while (j_3002 < n2_3002) {
44            arr_3002[k_3002] = R_3002[j_3002];
45            j_3002++;
46            k_3002++;
47        }
48    }
49    void sort(int arr_3002[], int l_3002, int r_3002) {
50        if (l_3002 < r_3002) {
51            // Find the middle point
52            int m_3002 = (l_3002 + r_3002) / 2;
53
54            // Sort first and second halves
55            sort(arr_3002, l_3002, m_3002);
56            sort(arr_3002, m_3002 + 1, r_3002);
57
58            // Merge the sorted halves
59            merge(arr_3002, l_3002, m_3002, r_3002);
60        }
61    }
62
63    /* A utility function to print array of size n */
64    static void printArray(int arr_3002[]) {
65        int n_3002 = arr_3002.length;
66        for (int i_3002 = 0; i_3002 < n_3002; ++i_3002)
67            System.out.print(arr_3002[i_3002] + " ");
68        System.out.println();
69    }
70
71    public static void main(String args_3002[]) {
72        int arr_3002[] = { 12, 11, 13, 5, 6, 7 };
73        System.out.println("Sebelum terurut");
```



```

74     printArray(arr_3002);
75     MergeSort_2511533002 ob_3002 = new MergeSort_2511533002();
76     ob_3002.sort(arr_3002, 0, arr_3002.length - 1);
77     System.out.println("\nSesudah Terurut menggunakan merge Sort");
78     printArray(arr_3002);
79 }
80 }

```

2.3.2 Uraian Kode Program

1. **int n1_3002 = m_3002 - l_3002 + 1;**
int n2_3002 = r_3002 - m_3002; = array akan dibagi menjadi dua bagian, yaitu bagian kiri dan bagian kanan.
2. **int L_3002[] = new int[n1_3002];**
int R_3002[] = new int[n2_3002]; = Array sementara ini digunakan agar data dari array utama bisa dibandingkan dan digabungkan kembali secara terurut.
3. **for (int i_3002 = 0; i_3002 < n1_3002; ++i_3002)**
L_3002[i_3002] = arr_3002[l_3002 + i_3002]; = Kode ini digunakan untuk menyalin data dari array utama ke array kiri
4. **for (int j_3002 = 0; j_3002 < n2_3002; ++j_3002)**
R_3002[j_3002] = arr_3002[m_3002 + 1 + j_3002]; = Kode ini digunakan untuk menyalin data dari array utama ke array kanan
5. **while (i_3002 < n1_3002 && j_3002 < n2_3002)** { = Perulangan ini berjalan selama array kiri dan array kanan masih memiliki elemen yang belum dimasukkan kembali ke array utama.
6. **if (L_3002[i_3002] <= R_3002[j_3002])** {
arr_3002[k_3002] = L_3002[i_3002];
i_3002++; = Jika elemen pada array kiri lebih kecil atau sama dengan elemen pada array kanan, maka elemen dari array kiri dimasukkan ke array utama.
7. **else** {
arr_3002[k_3002] = R_3002[j_3002];
j_3002++; = Jika elemen pada array kanan lebih kecil, maka elemen dari array kanan dimasukkan ke array utama.
8. **while (i_3002 < n1_3002)** {
arr_3002[k_3002] = L_3002[i_3002];
i_3002++;
k_3002++; = Kode ini digunakan untuk menyalin sisa elemen dari array kiri jika masih ada elemen yang belum dimasukkan ke array utama.
9. **while (j_3002 < n2_3002)** {
arr_3002[k_3002] = R_3002[j_3002];
j_3002++;
k_3002++; = Kode ini digunakan untuk menyalin sisa elemen dari array kanan jika masih ada elemen yang belum dimasukkan ke array utama.
10. **merge(arr_3002, l_3002, m_3002, r_3002);** = method merge dipanggil untuk menggabungkan kedua bagian tersebut menjadi satu array yang terurut.
11. **for (int i_3002 = 0; i_3002 < n_3002; ++i_3002)**

`System.out.print(arr_3002[i_3002] + " ");` = Perulangan ini digunakan untuk mencetak setiap elemen array satu per satu.

2.3.3 Output

```
Sebelum terurut
12 11 13 5 6 7

Sesudah Terurut menggunakan merge Sort
5 6 7 11 12 13
```

2.4 Merge Sort GUI

2.4.1 Kode Program

```
22 public class MergeSortGUI_2511533002 extends JFrame {
23     private static final long serialVersionUID = 1L;
24     private int[] array_3002;
25     private JLabel[] labelArray_3002;
26     private JButton stepButton_3002, resetButton_3002, setButton_3002;
27     private JTextField inputField_3002;
28     private JPanel panelArray_3002;
29     private JTextArea stepArea_3002;
30
31     // Mempertahankan i dan j tanpa akhiran sesuai permintaan sebelumnya
32     private int i, j;
33
34     // Variabel yang kurang ditambahkan di sini
35     private boolean sorting_3002 = false;
36     private int stepCount_3002 = 1;
37     private Queue<int[]> mergeQueue_3002 = new LinkedList<>();
38     private boolean isMerging_3002 = false;
39     private boolean copying_3002 = false;
40     private int left_3002, mid_3002, right_3002, k_3002;
41     private int[] temp_3002;
42
43     /**
44      * Create the frame.
45      */
46     public MergeSortGUI_2511533002() {
47         setTitle("Merge Sort Langkah per Langkah"); // Diubah agar sesuai
48         setSize(750, 400);
49         setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
50         setLocationRelativeTo(null);
51         setLayout(new BorderLayout());
52
53         JPanel inputPanel = new JPanel(new FlowLayout());
54         inputField_3002 = new JTextField(30);
55         setButton_3002 = new JButton("Set Array");
56         inputPanel.add(new JLabel("Masukkan angka (pisahkan dengan koma) :"));
57
58         inputPanel.add(inputField_3002);
59         inputPanel.add(setButton_3002);
60
61         panelArray_3002 = new JPanel();
62         panelArray_3002.setLayout(new FlowLayout());
63         JPanel controlPanel_3002 = new JPanel();
64         stepButton_3002 = new JButton("Langkah Selanjutnya");
65         resetButton_3002 = new JButton("Reset");
66         stepButton_3002.setEnabled(false);
67         controlPanel_3002.add(stepButton_3002);
68         controlPanel_3002.add(resetButton_3002);
69         stepArea_3002 = new JTextArea(8, 60);
70         stepArea_3002.setEditable(false);
71         stepArea_3002.setFont(new Font("Monospaced", Font.PLAIN, 14));
72         JScrollPane scrollPane_3002 = new JScrollPane(stepArea_3002);
73
74         add(inputPanel, BorderLayout.NORTH);
75         add(panelArray_3002, BorderLayout.CENTER);
76         add(controlPanel_3002, BorderLayout.SOUTH);
77         add(scrollPane_3002, BorderLayout.EAST);
78
79         setButton_3002.addActionListener(e -> setArrayFromInput_3002());
80         stepButton_3002.addActionListener(e -> performStep_3002());
81         resetButton_3002.addActionListener(e -> reset_3002());
82     }
83
84     private void setArrayFromInput_3002() {
85         String text_3002 = inputField_3002.getText().trim();
86         if (text_3002.isEmpty()) return;
87         String[] parts_3002 = text_3002.split(",");
88         array_3002 = new int[parts_3002.length];
89         try {
90             for (int k = 0; k < parts_3002.length; k++) {
91                 array_3002[k] = Integer.parseInt(parts_3002[k].trim());
92             }
93         } catch (NumberFormatException e_3002) {
```

```

93         JOptionPane.showMessageDialog(this, "Masukkan hanya angka!",
94             "Error", JOptionPane.ERROR_MESSAGE);
95     return;
96 }
97
98 labelArray_3002 = new JLabel[array_3002.length];
99 panelArray_3002.removeAll();
100 for (int k = 0; k < array_3002.length; k++) {
101     labelArray_3002[k] = new JLabel(String.valueOf(array_3002[k]));
102     labelArray_3002[k].setFont(new Font("Arial", Font.BOLD, 24));
103     labelArray_3002[k].setOpaque(true);
104     labelArray_3002[k].setBackground(Color.WHITE);
105     labelArray_3002[k].setBorder(BorderFactory.createLineBorder(Color.BLACK));
106     labelArray_3002[k].setPreferredSize(new Dimension(50, 50));
107     labelArray_3002[k].setHorizontalAlignment(SwingConstants.CENTER);
108     panelArray_3002.add(labelArray_3002[k]);
109 }
110
111 mergeQueue_3002.clear();
112 generateMergeSteps_3002(0, array_3002.length - 1);
113 stepButton_3002.setEnabled(true);
114 stepArea_3002.setText("");
115 stepCount_3002 = 1;
116 isMerging_3002 = false;
117 panelArray_3002.revalidate();
118 panelArray_3002.repaint();
119 }
120
121 // Metode yang kurang ditambahkan untuk mencatat langkah-langkah merge sort ke antrean
122 private void generateMergeSteps_3002(int l_3002, int r_3002) {
123     if (l_3002 < r_3002) {
124         int m_3002 = (l_3002 + r_3002) / 2;
125         generateMergeSteps_3002(l_3002, m_3002);
126         generateMergeSteps_3002(m_3002 + 1, r_3002);
127         mergeQueue_3002.add(new int[]{l_3002, m_3002, r_3002});

```

```

131     private void performStep_3002() {
132         resetHighlights_3002();
133
134         if (!isMerging_3002 && !mergeQueue_3002.isEmpty()) {
135             int[] range_3002 = mergeQueue_3002.poll();
136             left_3002 = range_3002[0];
137             mid_3002 = range_3002[1];
138             right_3002 = range_3002[2];
139             temp_3002 = new int[right_3002 - left_3002 + 1];
140             i = left_3002;
141             j = mid_3002 + 1;
142             k_3002 = 0;
143             copying_3002 = false;
144             isMerging_3002 = true;
145             stepArea_3002.append("Langkah " + stepCount_3002++ +
146                 ": Mulai merge dari " + left_3002 + " ke " + right_3002 + "\n");
147             return;
148         }
149
150         if (isMerging_3002 && !copying_3002) {
151             if (i <= mid_3002 && j <= right_3002) {
152                 labelArray_3002[i].setBackground(color.CYAN);
153                 labelArray_3002[j].setBackground(color.CYAN);
154                 if (array_3002[i] <= array_3002[j]) {
155                     temp_3002[k_3002++] = array_3002[i++];
156                 } else {
157                     temp_3002[k_3002++] = array_3002[j++];
158                 }
159                 stepArea_3002.append("Langkah " + stepCount_3002++ + ": Bandingkan dan salin elemen\n");
160                 return;
161             } else if (i <= mid_3002) {
162                 temp_3002[k_3002++] = array_3002[i++];
163                 stepArea_3002.append("Langkah " + stepCount_3002++ + ": Salin sisa kiri\n");
164                 return;
165             } else if (j <= right_3002) {

```

```

166                 temp_3002[k_3002++] = array_3002[j++];
167                 stepArea_3002.append("Langkah " + stepCount_3002++ + ": Salin sisa kanan\n");
168                 return;
169             } else {
170                 copying_3002 = true;
171                 k_3002 = 0;
172                 return;
173             }
174         }
175
176         if (copying_3002 && k_3002 < temp_3002.length) {
177             array_3002[left_3002 + k_3002] = temp_3002[k_3002];
178             labelArray_3002[left_3002 + k_3002].setText(String.valueOf(temp_3002[k_3002]));
179             labelArray_3002[left_3002 + k_3002].setBackground(Color.GREEN);
180             k_3002++;
181             stepArea_3002.append("Langkah " + stepCount_3002++ + ": Tempelkan ke array utama\n");
182             return;
183         }
184
185         if (copying_3002 && k_3002 == temp_3002.length) {
186             isMerging_3002 = false;
187             copying_3002 = false;
188         }
189
190         if (mergeQueue_3002.isEmpty() && !isMerging_3002) {
191             stepArea_3002.append("Selesai.\n");
192             stepButton_3002.setEnabled(false);
193             JOptionPane.showMessageDialog(this, "Merge Sort selesai!");
194         }
195     }
196
197     private void updateLabels_3002() {
198         for (int k_3002 = 0; k_3002 < array_3002.length; k_3002++) {
199             labelArray_3002[k_3002].setText(String.valueOf(array_3002[k_3002]));
200         }

```

```

203 private void resetHighlights_3002() {
204     if (labelArray_3002 == null) return;
205     for (JLabel label_3002 : labelArray_3002) {
206         label_3002.setBackground(color.WHITE);
207     }
208 }
209
210 private void reset_3002() {
211     inputField_3002.setText("");
212     panelArray_3002.removeAll();
213     panelArray_3002.revalidate();
214     panelArray_3002.repaint();
215     stepArea_3002.setText("");
216     stepButton_3002.setEnabled(false);
217     if (mergeQueue_3002 != null) {
218         mergeQueue_3002.clear();
219     }
220     isMerging_3002 = false;
221     stepCount_3002 = 1;
222 }
223
224 private String arrayToString_3002(int[] arr_3002) {
225     StringBuilder sb_3002 = new StringBuilder();
226     for (int k_3002 = 0; k_3002 < arr_3002.length; k_3002++) {
227         sb_3002.append(arr_3002[k_3002]);
228         if (k_3002 < arr_3002.length - 1) sb_3002.append(", ");
229     }
230     return sb_3002.toString();
231 }
232
233 public static void main(String[] args) {
234     SwingUtilities.invokeLater(() -> {
235         MergeSortGUI_2511533002 gui = new MergeSortGUI_2511533002();
236         gui.setVisible(true);
237     });

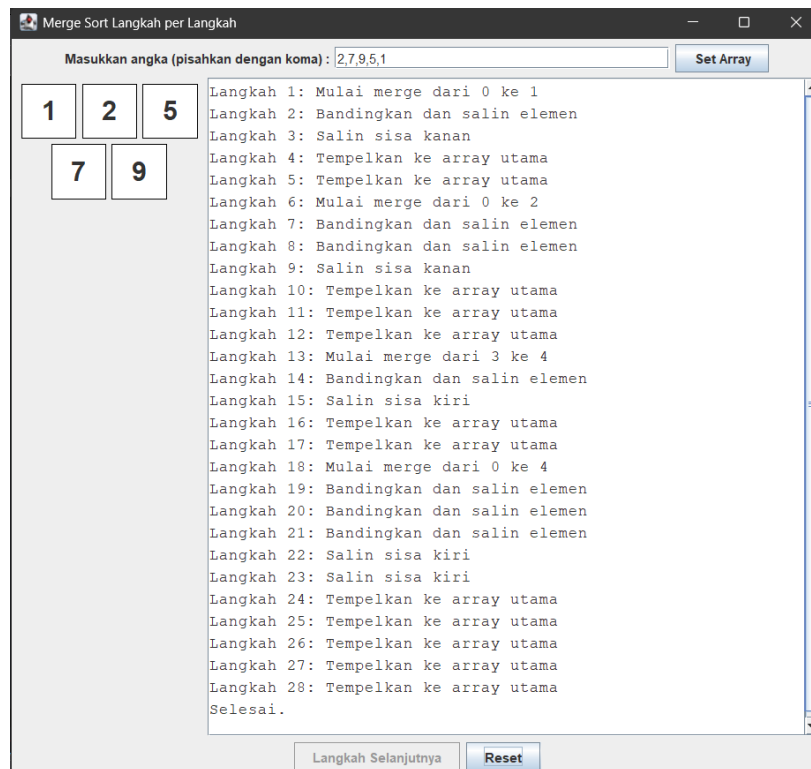
```

2.4.2 Uraian Kode Program

1. **array_3002 = new int[parts_3002.length];** = Kode ini digunakan untuk membuat array bertipe integer dengan ukuran sesuai jumlah angka yang dimasukkan pengguna.
2. **try {**
 for (int k = 0; k < parts_3002.length; k++) {
 array_3002[k] = Integer.parseInt(parts_3002[k].trim()); = Bagian ini digunakan untuk mengubah setiap data input dari bentuk String menjadi integer.
 catch (NumberFormatException e_3002) {
 JOptionPane.showMessageDialog(this, "Masukkan hanya angka!",
 "Error", JOptionPane.ERROR_MESSAGE);
 return; = Bagian catch akan dijalankan jika pengguna memasukkan data yang bukan angka.
 panelArray_3002.removeAll(); = Kode ini digunakan untuk menghapus semua tampilan angka sebelumnya pada panel.
 for (int k = 0; k < array_3002.length; k++) { = Perulangan ini digunakan untuk membuat label untuk setiap angka dalam array.
 labelArray_3002[k] = new JLabel(String.valueOf(array_3002[k])); = Kode ini membuat label baru yang berisi nilai dari array.
 generateMergeSteps_3002(0, array_3002.length - 1); = Kode ini memanggil method generateMergeSteps_3002() untuk membuat daftar langkah penggabungan pada Merge Sort.
 stepCount_3002 = 1; = Kode ini mengatur nomor langkah kembali ke angka 1.
 if (!isMerging_3002 && !mergeQueue_3002.isEmpty()) { = Kondisi ini berarti program belum sedang melakukan proses merge dan masih ada data langkah merge di dalam mergeQueue_3002.
 int[] range_3002 = mergeQueue_3002.poll(); = digunakan untuk mengambil satu data langkah merge dari antrian mergeQueue_3002.
 if (i <= mid_3002 && j <= right_3002) { = Kode ini mengecek apakah elemen pada bagian kiri dan bagian kanan masih tersedia untuk dibandingkan.
 if (array_3002[i] <= array_3002[j]) {
 temp_3002[k_3002++] = array_3002[i++]; = Jika nilai pada bagian kiri lebih kecil atau sama dengan nilai pada bagian kanan, maka nilai dari bagian kiri dimasukkan ke array sementara.

13. **else {**
 temp_3002[k_3002++] = array_3002[j++]; = Jika nilai pada bagian kanan lebih kecil, maka nilai dari bagian kanan dimasukkan ke array sementara.
14. **else if (i <= mid_3002) {**
 temp_3002[k_3002++] = array_3002[i++];
 stepArea_3002.append("Langkah " + stepCount_3002++ + ": Salin sisa kiri\n");
 return; = Bagian ini dijalankan jika elemen pada bagian kanan sudah habis, tetapi bagian kiri masih memiliki sisa elemen.
15. **else {**
 copying_3002 = true;
 k_3002 = 0;
 return; = Bagian ini dijalankan jika semua elemen dari bagian kiri dan kanan sudah selesai dimasukkan ke array sementara.
16. **labelArray_3002[left_3002 +**
 k_3002].setText(String.valueOf(temp_3002[k_3002])); = Kode ini memperbarui tampilan label agar sesuai dengan nilai terbaru pada array.
17. **for (int k_3002 = 0; k_3002 < array_3002.length; k_3002++)** = Perulangan ini berjalan dari indeks pertama sampai indeks terakhir array.
18. **labelArray_3002[k_3002].setText(String.valueOf(array_3002[k_3002]));** = Kode ini mengambil nilai dari array_3002, lalu menampilkannya ke dalam **labelArray_3002**.
19. **if (mergeQueue_3002 != null) {**
 mergeQueue_3002.clear(); = Kode ini mengecek apakah mergeQueue_3002 tidak bernilai null. Jika antrean tersebut ada, maka semua isinya dihapus
20. **for (int k_3002 = 0; k_3002 < arr_3002.length; k_3002++)** = Perulangan ini digunakan untuk membaca semua elemen array dari awal sampai akhir.
21. **StringBuilder sb_3002 = new StringBuilder();** = StringBuilder digunakan untuk menyusun teks secara bertahap.

2.4.3 Output



2.5 Bubble Sort GUI

2.5.1 Kode Program

```
20 public class BubbleSortGUI_2511533002 extends JFrame {
21     private static final long serialVersionUID = 1L;
22     private int[] array_3002;
23     private JLabel[] labelArray_3002;
24     private JButton stepButton_3002, resetButton_3002, setButton_3002;
25     private JTextField inputField_3002;
26     private JPanel panelArray_3002;
27     private JTextArea stepArea_3002;
28
29     private int i = 1, j;
30     private boolean sorting_3002 = false;
31     private int stepCount_3002 = 1;
32
33     /**
34      * Create the frame.
35      */
36     public BubbleSortGUI_2511533002() {
37         setTitle("Bubble Sort Langkah per Langkah");
38         setSize(750, 400);
39         setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
40         setLocationRelativeTo(null);
41         setLayout(new BorderLayout());
42
43         JPanel inputPanel = new JPanel(new FlowLayout());
44         inputField_3002 = new JTextField(30);
45         setButton_3002 = new JButton("Set Array");
46         inputPanel.add(new JLabel("Masukkan angka (pisahkan dengan koma) :"));
47         inputPanel.add(inputField_3002);
48         inputPanel.add(setButton_3002);
49
50         panelArray_3002 = new JPanel();
51         panelArray_3002.setLayout(new FlowLayout());
52         JPanel controlPanel_3002 = new JPanel();
53         stepButton_3002 = new JButton("Langkah Selanjutnya");
54         resetButton_3002 = new JButton("Reset");
```

```

55     stepButton_3002.setEnabled(false);
56     controlPanel_3002.add(stepButton_3002);
57     controlPanel_3002.add(resetButton_3002);
58     stepArea_3002 = new JTextArea(8, 60);
59     stepArea_3002.setEditable(false);
60     stepArea_3002.setFont(new Font("Monospaced", Font.PLAIN, 14));
61     JScrollPane scrollPane_3002 = new JScrollPane(stepArea_3002);
62     add(inputPanel, BorderLayout.NORTH);
63     add(panelArray_3002, BorderLayout.CENTER);
64     add(controlPanel_3002, BorderLayout.SOUTH);
65     add(scrollPane_3002, BorderLayout.EAST);
66     stepButton_3002.addActionListener(e -> setArrayFromInput_3002());
67     stepButton_3002.addActionListener(e -> performStep_3002());
68     resetButton_3002.addActionListener(e -> reset_3002());
69 }
70
71 private void setArrayFromInput_3002() {
72     String text_3002 = inputField_3002.getText().trim();
73     if (text_3002.isEmpty()) return;
74     String[] parts_3002 = text_3002.split(",");
75     array_3002 = new int[parts_3002.length];
76
77     try {
78         for (int k_3002 = 0; k_3002 < parts_3002.length; k_3002++) {
79             array_3002[k_3002] = Integer.parseInt(parts_3002[k_3002].trim());
80         }
81     } catch (NumberFormatException e_3002) {
82         JOptionPane.showMessageDialog(this, "Masukkan hanya angka "
83             + "yang dipisahkan koma!", "Error", JOptionPane.ERROR_MESSAGE);
84         return;
85     }
86
87     i = 0;
88     j = 0;

```

```

89     stepCount_3002 = 1;
90     sorting_3002 = true;
91     stepButton_3002.setEnabled(true);
92     stepArea_3002.setText("");
93     panelArray_3002.removeAll();
94     labelArray_3002 = new JLabel[array_3002.length];
95
96     for (int k_3002 = 0; k_3002 < array_3002.length; k_3002++) {
97         labelArray_3002[k_3002] = new JLabel(String.valueOf(array_3002[k_3002]));
98         labelArray_3002[k_3002].setFont(new Font("Arial", Font.BOLD, 24));
99         labelArray_3002[k_3002].setOpaque(true);
100        labelArray_3002[k_3002].setBackground(Color.WHITE);
101        labelArray_3002[k_3002].setBorder(BorderFactory.createLineBorder(Color.BLACK));
102        labelArray_3002[k_3002].setPreferredSize(new Dimension(50, 50));
103        labelArray_3002[k_3002].setHorizontalAlignment(SwingConstants.CENTER);
104        panelArray_3002.add(labelArray_3002[k_3002]);
105    }
106
107    panelArray_3002.revalidate();
108    panelArray_3002.repaint();
109 }
110
111 private void performStep_3002() {
112     if (!sorting_3002 || i >= array_3002.length - 1) {
113         sorting_3002 = false;
114         stepButton_3002.setEnabled(false);
115         JOptionPane.showMessageDialog(this, "Sorting selesai!");
116         return;
117     }
118     resetHighlights_3002();
119
120     StringBuilder stepLog_3002 = new StringBuilder();
121     labelArray_3002[j].setBackground(Color.CYAN);
122     labelArray_3002[j + 1].setBackground(Color.CYAN);
123
124     if (array_3002[j] > array_3002[j + 1]) {

```

```

125         int temp_3002 = array_3002[j];
126         array_3002[j] = array_3002[j + 1];
127         array_3002[j + 1] = temp_3002;
128
129         labelArray_3002[j].setBackground(Color.RED);
130         labelArray_3002[j + 1].setBackground(Color.RED);
131
132         stepLog_3002.append("Langkah ").append(stepCount_3002).append(": Menukar elemen ke-")
133             .append(j).append(" ").append(array_3002[j + 1]).append(" dengan ke-")
134             .append(j + 1).append(" ").append(array_3002[j]).append("\n");
135     } else {
136         stepLog_3002.append("Langkah ").append(stepCount_3002).append(": Tidak ada pertukaran antara ke-")
137             .append(j).append(" dan ke-").append(j + 1).append("\n");
138     }
139
140     stepLog_3002.append("Hasil: ").append(arrayToString_3002(array_3002)).append("\n\n");
141     stepArea_3002.append(stepLog_3002.toString());
142
143     updateLabels_3002();
144     j++;
145
146     if (j >= array_3002.length - i - 1) {
147         j = 0;
148         i++;
149     }
150
151     stepCount_3002++;
152
153     if (i >= array_3002.length - 1) {
154         sorting_3002 = false;
155         stepButton_3002.setEnabled(false);
156         JOptionPane.showMessageDialog(this, "Sorting selesai!");
157     }
158 }

```

```

161 private void updateArray_3002() {
162     for (int k_3002 = 0; k_3002 < array_3002.length; k_3002++) {
163         labelArray_3002[k_3002].setText(String.valueOf(array_3002[k_3002]));
164     }
165 }
166 private void resetHighlights_3002() {
167     for (JLabel label_3002 : labelArray_3002) {
168         label_3002.setBackground(Color.WHITE);
169     }
170 }
171 }
172 private void reset_3002() {
173     inputField_3002.setText("");
174     panelArray_3002.removeAll();
175     panelArray_3002.revalidate();
176     panelArray_3002.repaint();
177     stepArea_3002.setText("");
178     stepButton_3002.setEnabled(false);
179     sorting_3002 = false;
180     i = 0;
181     j = 0;
182     stepCount_3002 = 1;
183 }
184 private String arrayToString_3002(int[] arr_3002) {
185     StringBuilder sb_3002 = new StringBuilder();
186     for (int k_3002 = 0; k_3002 < arr_3002.length; k_3002++) {
187         sb_3002.append(arr_3002[k_3002]);
188         if (k_3002 < arr_3002.length - 1) sb_3002.append(", ");
189     }
190     return sb_3002.toString();
191 }
192 }
193 public static void main(String[] args) {
194     SwingUtilities.invokeLater(() -> {
195
196         BubbleSortGUI_2511533002 gui = new BubbleSortGUI_2511533002();
197         gui.setVisible(true);
198     });
199 }
200 }

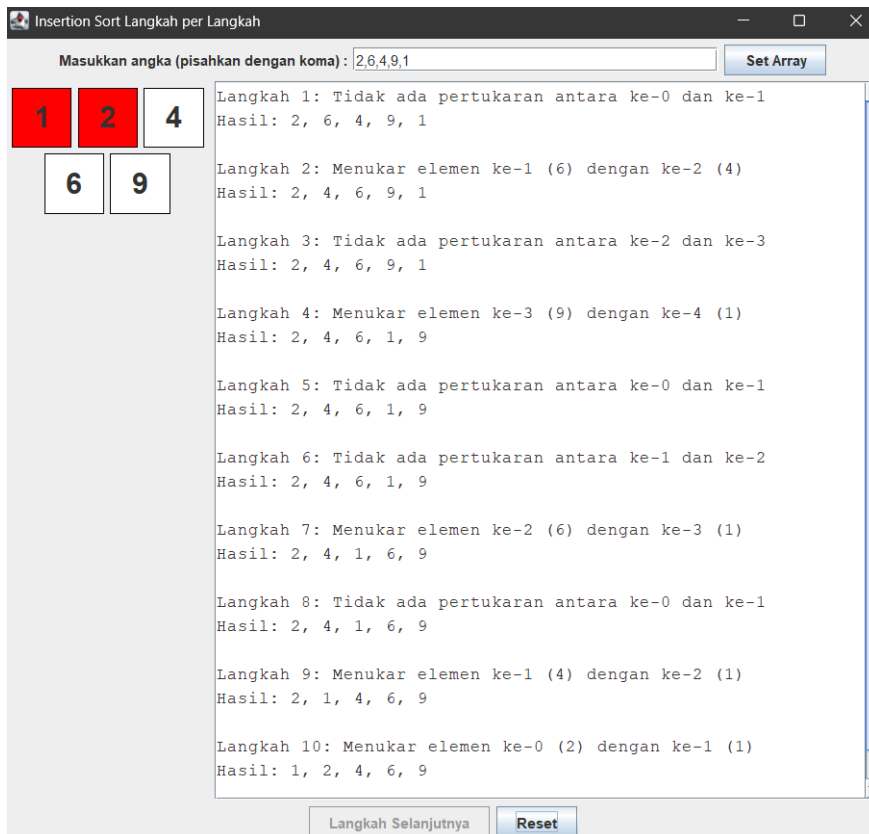
```

2.5.2 Uraian Kode Program

1. **array_3002 = new int[parts_3002.length];** = membuat array integer dengan ukuran sesuai jumlah angka yang dimasukkan pengguna.
2. **try {**
for (int k_3002 = 0; k_3002 < parts_3002.length; k_3002++) {
array_3002[k_3002] = Integer.parseInt(parts_3002[k_3002].trim()); = Bagian ini digunakan untuk mengubah setiap data dari bentuk String menjadi integer.
3. **for (int k_3002 = 0; k_3002 < parts_3002.length; k_3002++)** = Perulangan ini membaca semua elemen dari parts_3002.
4. **parts_3002[k_3002].trim()** = Digunakan untuk menghapus spasi pada setiap angka.
5. **i = 0;**
j = 0;
stepCount_3002 = 1;
sorting_3002 = true; = Kode ini digunakan untuk mengatur ulang variabel yang berhubungan dengan proses sorting.
6. **labelArray_3002 = new JLabel[array_3002.length];** = Kode ini membuat array label dengan jumlah yang sama seperti jumlah elemen pada array_3002.
7. **for (int k_3002 = 0; k_3002 < array_3002.length; k_3002++) {** = Perulangan ini digunakan untuk membuat label untuk setiap angka dalam array.
8. **StringBuilder stepLog_3002 = new StringBuilder();** = StringBuilder digunakan untuk menyusun teks keterangan langkah sorting.
9. **if (array_3002[j] > array_3002[j + 1]) {** = Kode ini mengecek apakah elemen kiri lebih besar daripada elemen kanan.
10. **int temp_3002 = array_3002[j];**
array_3002[j] = array_3002[j + 1];
array_3002[j + 1] = temp_3002; = Kode ini digunakan untuk menukar posisi dua elemen.

11. `stepArea_3002.append(stepLog_3002.toString());` = Kode ini menampilkan isi `stepLog_3002` ke dalam `stepArea_3002`.
12. `updateLabels_3002();` = Kode ini digunakan untuk memperbarui angka pada label GUI sesuai dengan isi array terbaru.
13. `j++;` = digunakan untuk berpindah ke pasangan elemen berikutnya.
14. `if (j >= array_3002.length - i - 1) {`
`j = 0;`
`i++;` = Kode ini digunakan untuk mengecek apakah satu putaran Bubble Sort sudah selesai.
15. `stepCount_3002++;` = digunakan untuk menaikkan nomor langkah.
16. `labelArray_3002[k_3002].setText(String.valueOf(array_3002[k_3002]));` = Kode ini mengubah teks pada label agar sesuai dengan nilai array terbaru.

2.5.3 Output



BAB III

TUGAS

3.1 Playlist Lagu

3.1.1 Kode Program

```
1 package Pekan8_2511533002;
2 class Lagu {
3     String judul_3002;
4     String penyanyi_3002;
5     int durasi_3002;
6
7     public Lagu(String judul, String penyanyi, int durasi) {
8         this.judul_3002 = judul;
9         this.penyanyi_3002 = penyanyi;
10        this.durasi_3002 = durasi;
11    }
12 }
13
14 public class Lagu_2511533002 {
15     Lagu[] dataLagu_3002 = new Lagu[20];
16     int jumlahLagu_3002 = 0;
17
18     public void inputData_3002() {
19         dataLagu_3002[jumlahLagu_3002++] = new Lagu("Let Go", "Centrell Cee", 270);
20         dataLagu_3002[jumlahLagu_3002++] = new Lagu("I Like You", "Post Malone", 252);
21         dataLagu_3002[jumlahLagu_3002++] = new Lagu("Silence", "Marshmellow", 196);
22         dataLagu_3002[jumlahLagu_3002++] = new Lagu("God's Plan", "Drake", 354);
23         dataLagu_3002[jumlahLagu_3002++] = new Lagu("See You Again", "Charlie Puth", 390);
24         dataLagu_3002[jumlahLagu_3002++] = new Lagu("Danza Kuduro", "Don Omar", 233);
25         dataLagu_3002[jumlahLagu_3002++] = new Lagu("Humble", "Skrillex", 200);
26    }
27
28     public void tampilData_3002(String pesan) {
29         System.out.println(pesan);
30         for (int i = 0; i < jumlahLagu_3002; i++) {
31             System.out.println((i + 1) + ". " + dataLagu_3002[i].judul_3002 + " - " + dataLagu_3002[i].durasi_3002 + " detik");
32         }
33         System.out.println();
34    }
35
36     public void shellSort_3002() {
37         int n = jumlahLagu_3002;
38
39         for (int gap = n / 2; gap > 0; gap /= 2) {
40             for (int i = gap; i < n; i++) {
41                 Lagu temp = dataLagu_3002[i];
42                 int j;
43                 for (j = i; j >= gap && dataLagu_3002[j - gap].judul_3002.compareToIgnoreCase(temp.judul_3002) > 0; j -= gap) {
44                     dataLagu_3002[j] = dataLagu_3002[j - gap];
45                 }
46                 dataLagu_3002[j] = temp;
47             }
48         }
49    }
50
51     public static void main(String[] args) {
52         Lagu_2511533002 playlist = new Lagu_2511533002();
53         playlist.inputData_3002();
54         System.out.println("=== Sorting Playlist NIM: 2511533002 ===");
55         playlist.tampilData_3002("Data Sebelum Sorting:");
56         playlist.shellSort_3002();
57         playlist.tampilData_3002("Data Setelah Shell Sort (Judul A-Z):");
58    }
59 }
```

3.1.2 Uraian Kode Program

1. **for (int i = 0; i < jumlahLagu_3002; i++)** { = Perulangan dari indeks 0 hingga sebelum jumlah total lagu yang ada.
2. **System.out.println((i + 1) + ". " + dataLagu_3002[i].judul + " - " + dataLagu_3002[i].durasi + " detik");** = "Nomor. Judul Lagu - Durasi detik". (i + 1) digunakan agar penomoran di layar dimulai dari 1, bukan 0.
3. **for (int gap = n / 2; gap > 0; gap /= 2)** { = Shell sort bekerja dengan membandingkan elemen yang berjarak jauh (disebut gap), lalu jaraknya diperkecil hingga jarak 1. Baris ini mengatur jarak awal sebesar setengah dari total data (n / 2), lalu dibagi 2 terus-menerus (gap /= 2) sampai gap lebih besar dari 0.
4. **for (int i = gap; i < n; i++)** { = Memulai pengecekan dari elemen di posisi gap hingga elemen terakhir.
5. **for (j = i; j >= gap && dataLagu_3002[j - gap].judul.compareToIgnoreCase(temp.judul) > 0; j -= gap)** { = Mundur membandingkan lagu dengan lagu sebelumnya yang berjarak gap.

6. `dataLagu_3002[j] = dataLagu_3002[j - gap];` = Karena judul sebelumnya lebih besar, posisinya digeser ke kanan (ke posisi j saat ini) untuk memberi ruang bagi lagu yang abjadnya lebih kecil.

3.2 Output

```
=== Sorting Playlist NIM: 2511533002 ===  
Data Sebelum Sorting:  
1. Let Go - 270 detik  
2. I LIke You - 252 detik  
3. Silence - 196 detik  
4. God's Plan - 354 detik  
5. See You Again - 390 detik  
6. Danza Kuduro - 233 detik  
7. Humble - 200 detik  
  
Data Setelah Shell Sort (Judul A-Z):  
1. Danza Kuduro - 233 detik  
2. God's Plan - 354 detik  
3. Humble - 200 detik  
4. I LIke You - 252 detik  
5. Let Go - 270 detik  
6. See You Again - 390 detik  
7. Silence - 196 detik
```

BAB III

KESIMPULAN

Berdasarkan praktikum yang telah dilakukan, dapat disimpulkan bahwa sorting merupakan proses penting dalam pemrograman karena digunakan untuk mengurutkan data agar lebih rapi dan mudah dikelola. Algoritma sorting seperti Quick Sort, Shell Sort, Merge Sort, dan Bubble Sort memiliki cara kerja yang berbeda-beda dalam menyusun data. Quick Sort bekerja dengan memilih pivot, Shell Sort menggunakan gap, Merge Sort membagi data menjadi bagian kecil lalu menggabungkannya kembali, sedangkan Bubble Sort menukar data secara berulang jika urutannya belum sesuai.

Penerapan sorting dalam bentuk GUI juga sangat membantu dalam memahami proses pengurutan data. Dengan adanya Merge Sort GUI dan Bubble Sort GUI, proses sorting tidak hanya dapat dilihat dari hasil akhirnya, tetapi juga dari tahapan-tahapan pengurutannya. Hal ini membuat mahasiswa lebih mudah memahami alur kerja algoritma dan membandingkan cara kerja masing-masing metode sorting.

Secara keseluruhan, praktikum ini memberikan pemahaman bahwa pemilihan algoritma sorting sangat penting dalam pembuatan program. Untuk data yang sedikit, algoritma sederhana seperti Bubble Sort masih dapat digunakan. Namun, untuk data yang lebih banyak, algoritma seperti Quick Sort, Shell Sort, dan Merge Sort lebih efektif karena memiliki proses pengurutan yang lebih efisien.